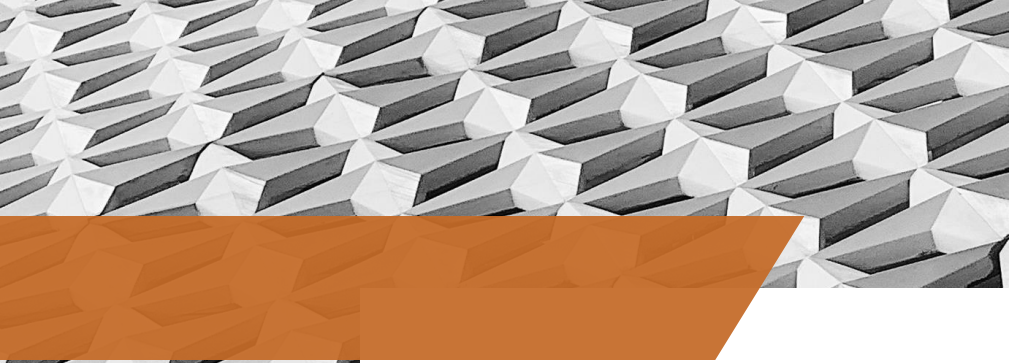
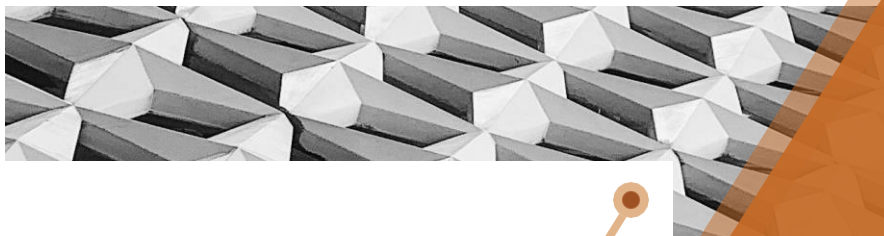






INTRODUCCIÓN AL DESARROLLO DE SOFTWARE



BIENVENIDA

Bienvenido(a) a la asignatura *Introducción al Desarrollo de Software*, con la cual podrás identificar cómo deben resolverse los problemas computacionales de los sistemas de información que desarrollarás. En general, lograrás reconocer pseudocódigo y algoritmos, así como la manera en que se manipulan los datos, su interacción con el usuario y paradigmas de programación. A su vez, comprenderás la creación de los diagramas de flujo.

LIBROS RECOMENDADOS

- Jain, H. (2017) Problem Solving in Data Structures & Algorithms Using C.

- Heineman, G. T., Pollice, G. & Selkow, S. (2016) Algorithms in a Nutshell: A practical guide. O'Reilly.



1 UNIDAD

RESOLUCIÓN DE PROBLEMAS COMPUTACIONALES



TEMARIO

1.1 PSEUDOCÓDIGO Y ALGORITMOS







1.2 DIAGRAMAS DE FLUJO



INTRODUCCIÓN

La asignatura *Introducción al Desarrollo de Software* tiene como objetivo comprender la creación de programas informáticos que satisfagan las necesidades de los usuarios.

En esta primera unidad aprenderás a desarrollar pseudocódigo y algoritmos, plasmándolos en diagramas de flujo.

COMPETENCIAS A DESARROLLAR



PSEUDOCÓDIGO Y ALGORITMOS

El **pseudocódigo** está constituido por una serie de palabras que describen los pasos que va a ejecutar un programa. Para ello, utiliza diversas expresiones, las cuales son matemáticas, lógicas y variables. También es conocido como lenguaje falso.

Su principal objetivo **es interpretar las instrucciones de la resolución de un algoritmo**. Además, el pseudocódigo debe ser similar al lenguaje con el que más tarde se codificará el programa.



PSEUDOCÓDIGO Y ALGORITMOS

En este contexto, el pseudocódigo sirve como un **instructivo no formal** en el que se pueden ver claramente los problemas de un programa.

De igual forma, funciona como una manera de transmitir las ideas a otras personas, ya que está pensado para facilitar el entendimiento de un algoritmo; sin embargo, puede omitir ciertos detalles.





PSEUDOCÓDIGO Y ALGORITMOS

Las **características principales del pseudocódigo** son:

- Representan algoritmos
- Pueden ser ejecutados en una computadora
- Siguen un diseño descendente (de arriba hacia abajo)

Los pseudocódigos tienen **tres tipos de estructuras de control**:

1. **Estructuras secuenciales.** Se ejecutan una tras otra. Si la anterior no ha finalizado no se puede continuar. Además, se ejecutan de arriba hacia abajo. Por ejemplo:

```
instrucción 1
instrucción 2
instrucción 3
.....
instrucción n
```

2. **Estructuras selectivas.** Se ejecutan según sea la condición dada, ya que señalan cuál es camino que se debe seguir alternativamente. Las condiciones son de tipo *booleano*. Las instrucciones se ejecutan solo si son verdaderas; si no, ejecutan otra instrucción. Por ejemplo:

```
if (condición)
        instrucción1;
else
        instrucción2;
```

Las estructuras selectivas pueden tener **varias condiciones** en el mismo cuerpo del pseudocódigo. Para ello, utilizan llaves "{ }" para determinar el segmento de instrucciones. Existen en **dos formas**:

- Sentencia *if-then-else* y
- Sentencia *switch*.

Sentencia *if-then-else*. Nos dice que si se cumple cierta condición, entonces se realiza dicha instrucción; si no sucede así, se realiza otra instrucción diferente.

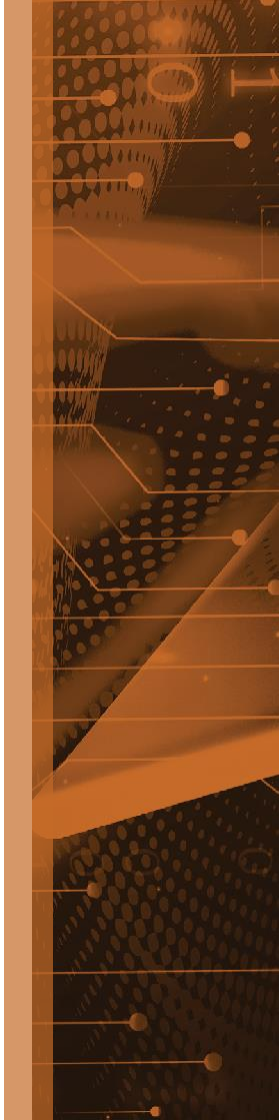
Ejemplo de sentencia *if-then-else*:

```
if (condición1) then
   {
        instrucción1;
        instrucción2;
   }
else if (condición2) then
   {
        instrucción3;
   }
else
{
        instrucción4;
}
end if
```

Sentencia *switch*. Ejecuta un trozo de código de acuerdo a cierto valor dado. Se trata de una opción múltiple, ya que el flujo del programa varía según la opción elegida al principio. Por ejemplo:

```
switch (variable)

 {
        case constante1: acción1;
              break;
        case constante1: acción2;
              break;
        case constante1: acción3;
              break;
        default: acción4;
 }
```



3. Estructuras iterativas. Se ejecuta varias veces el mismo trozo de código hasta que sea necesario. Estas estructuras también son llamadas bucles o ciclos. Dentro de estas, existen **diferentes tipos**, como son:

a) *for.* Se define un bucle dominado por un contador que se ejecuta una sola vez. Enseguida se designa el contador hasta donde debe cumplirse. Por último, se muestra la actualización del contador.

Ejemplo de estructura *for*:

```
for (...; ...; ...)
 {
        acción1;
        acción2;
        acción3;
        acción4;
        ..............
 }
```

Podemos observar un bucle que **repite un conjunto de instrucciones.**

b) ***while.*** Se repite el bloque de código hasta que la condición establecida se efectúe, y luego se evalúa la condición. Por ejemplo:

```
while (condición)
 {
        instrucción 2;
        instrucción 3;
        instrucción 4;
        ..................
 }
```

c) *do while.* En esta estructura se ejecuta un trozo de código solo cuando una condición se cumpla. Primero se lee todo el código una vez, y después se evalúa la condición. Por ejemplo:

```
do
 {
        acción 1;
        acción 2;
        acción 3;
        acción 4;
        ..............
 } while (condición);
```

PSEUDOCÓDIGO Y ALGORITMOS

Un **algoritmo** es una serie finita de instrucciones dadas para resolver un problema, efectuar un cálculo o realizar una actividad. En programación, representa un paso previo a la realización del código.

Los algoritmos se representan gráficamente mediante diagramas de flujo.

¿Sabías que todas las tareas que se realizan tanto en programación como en la vida cotidiana están basadas en algoritmos?





PSEUDOCÓDIGO Y ALGORITMOS

El **objetivo principal de los algoritmos** es el de resolver o dar una solución a un problema de manera clara, para que pueda ser desarrollado en cualquier lenguaje de programación.

Las **características** de un algoritmo son:

- Es finito (con inicio y fin)
- Tiene una secuencia
- Es preciso
- Tiene lógica

PSEUDOCÓDIGO Y ALGORITMOS

Las partes de un algoritmo son:

- ***Input* (entrada).** Representa la información que se introduce para trabajar con ella.
- **Proceso.** Serie de pasos que se seguirán para darle solución al problema.
- ***Output* (salida).** Resultados que se obtienen del algoritmo.

Todo algoritmo debe seguir estos tres pasos en el orden antes mencionado. De lo contrario, el algoritmo no será correcto ni podrá tener solución.





Etapas de desarrollo de un algoritmo:

- **Definición.** Se define el problema que se debe resolver.
- **Análisis.** Se analiza el problema completo. A su vez, se busca la solución más adecuada.
- **Diseño.** Se modela la solución al problema. Para ello, se busca una herramienta de diseño para la creación del diagrama de flujo.
- **Implementación.** Aquí se concreta el programa.

PSEUDOCÓDIGO Y ALGORITMOS

Ejemplo de un algoritmo sencillo:

Algoritmo para la suma de dos valores:

Inicio
Ingresar el valor “1”
Ingresar el valor “2”
Suma = valor “1” + valor “2”
Imprimir *Suma*
Fin

Como podemos observar, se pueden crear diversos algoritmos. Estos pueden ser de programación o para resolver cualquier problema.

PSEUDOCÓDIGO Y ALGORITMOS

Diferencia principal entre pseudocódigo y algoritmo:

- El pseudocódigo es una forma de representar al algoritmo de una manera entendible para las personas, quizás omitiendo pasos, pero siendo más entendible.

- El algoritmo debe ser preciso y finito, explicando detalladamente cada paso sin omitir nada. De lo contrario, el problema no tendría una óptima resolución.



VIDEO



Te invitamos a ver el siguiente video:

DIAGRAMAS DE FLUJO

Los **diagramas de flujo** son la representación gráfica de un algoritmo. Simbolizan un proceso con el apoyo de distintas figuras específicas, las cuales son sus partes fundamentales.

Su objetivo principal es ordenar, de la mejor forma, los pasos a seguir del algoritmo. De esta manera, la principal ventaja de los diagramas de flujo es que pueden almacenar mucha información, la cual es representada de una manera más clara para su máximo entendimiento.





DIAGRAMAS DE FLUJO

Características principales de los diagramas de flujo:

- Se componen de bloques de información con flechas que indican el flujo del proceso.
- Tienen un inicio y un fin.
- Se simbolizan para su mejor entendimiento.
- Permiten leer un proceso extenso de manera rápida.
- Sirven como herramienta de control para organizar información.

DIAGRAMAS DE FLUJO

Normas para realizar un diagrama de flujo:

1. Identificar las ideas principales.
2. Definir objetivos.
3. Identificar el tipo de diagrama de flujo que se creará.
4. Elegir los detalles y límites que tendrá.

Gracias a los diagramas de flujo, hoy en día la información de un proceso puede transmitirse a diferentes personas, ya que facilitan el entendimiento del problema que se desea resolver.



DIAGRAMAS DE FLUJO

Simbología para crear diagramas de flujo:



Inicio o fin del diagrama



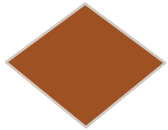
Proceso



Línea de flujo (indica el orden del proceso)



Entrada de datos



Decisión (es donde se analiza si es verdadero o falso)

DIAGRAMAS DE FLUJO



Base de datos



Conector en misma página



Imprimir resultados



Subprograma



Conector fuera de página

DIAGRAMAS DE FLUJO

Pasos para crear un diagrama de flujo:

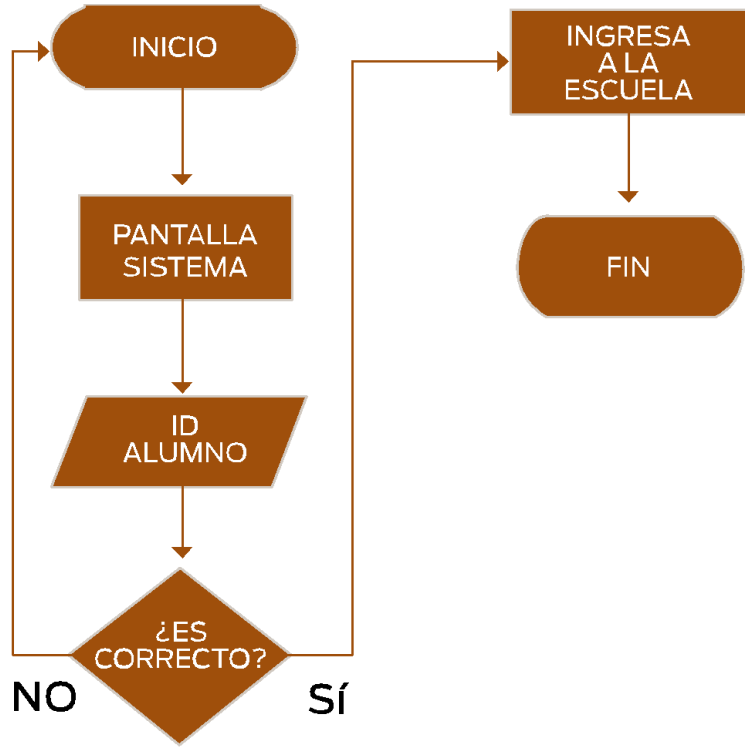
1. Determinar cuáles serán las entradas y salidas del proceso.
2. Ordenar la información que se tiene para poder plasmarla.
3. Elegir la simbología adecuada para su creación.
4. Identificar puntos de decisión.
5. Construir el diagrama respetando los lineamientos establecidos.
6. Probar el diagrama para verificar que no existan errores.



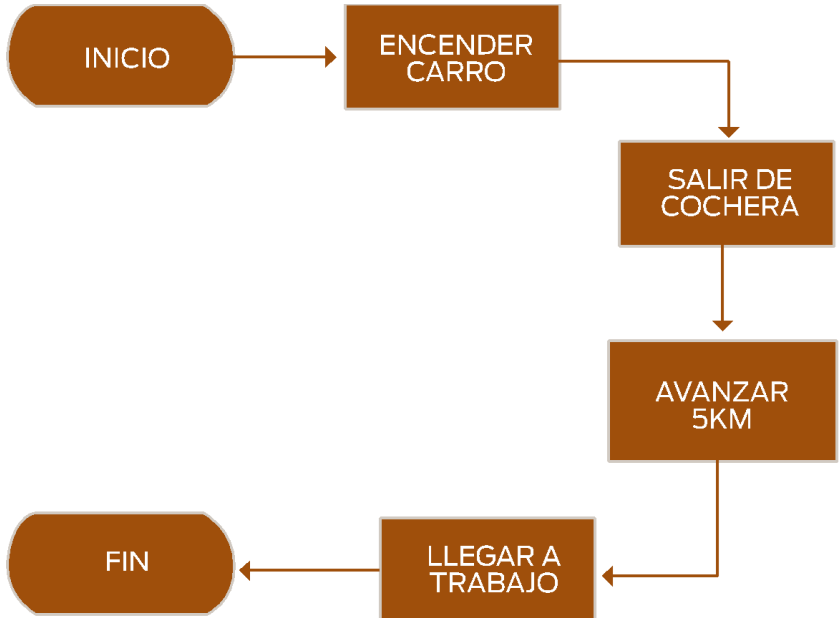
DIAGRAMAS DE FLUJO

Tipos de diagrama de flujo:

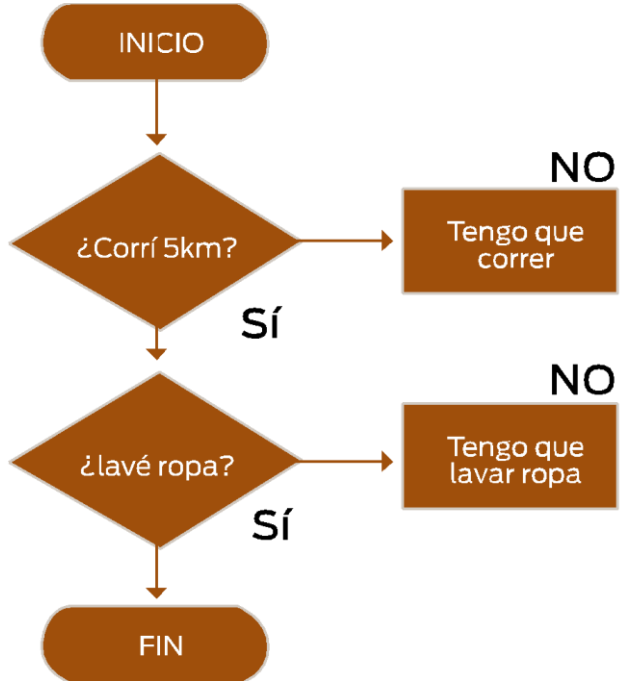
1. **Formato vertical.** En este tipo de diagrama, el flujo del proceso va de arriba hacia abajo. Por ejemplo:



2. Formato horizontal. En este tipo de diagrama de flujo, el proceso va de izquierda a derecha. Por ejemplo:



3. Formato panorámico. En este tipo de diagrama de flujo, el proceso está plasmado en un solo diagrama. Así, se toma tanto el sentido horizontal como el vertical para representar diferentes acciones.



FORO 1

Entorno de trabajo.

Participa en el foro enviando imágenes que demuestren que ya tienes acceso a las siguientes herramientas en su versión de prueba:

- Programiz

Presiona el botón para participar en el foro.

CONCLUSIÓN

Durante esta unidad comprendimos cómo son creados los pseudocódigos y algoritmos desde cero, para después plasmarlos en su forma gráfica, mediante diagramas de flujo. Además, revisamos algunos ejemplos prácticos para comprender la composición de los diagramas de flujo, así como la reglas para su creación.

En definitiva, es muy importante resaltar que la clara comprensión de estos temas favorece a nuestro conocimiento en programación. En este sentido, su uso facilita la utilización del código de algún lenguaje de programación. De la misma manera, nos permite abordar los problemas para poder encontrar una solución favorable a la hora de su resolución.



¡FELICIDADES!

Acabas de concluir la primera unidad de tu curso *Introducción al Desarrollo de Software*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

2

UNIDAD

MANIPULACIÓN DE DATOS



TEMARIO



2.1 VARIABLES Y CONSTANTES



2.2 TIPOS DE DATOS



INTRODUCCIÓN

En esta segunda unidad aprenderás a distinguir entre las variables y las constantes. De la misma manera, conocerás los distintos tipos de datos que existen. Esta distinción te permitirá crear el código de programación de manera correcta, al saber identificar qué se necesita utilizar según lo requerido.

COMPETENCIAS A DESARROLLAR



El alumno identificará de manera correcta las variables y las constantes.

El alumno conocerá y comprenderá los distintos tipos de datos existentes.

VARIABLES Y CONSTANTES

Existen dos tipos de datos simples, a saber, las **variables** y las **constantes**:

- **Variable.** Es un espacio en memoria donde se puede introducir un dato. Después, este se pasa y se transforma. De esta manera, podemos mostrar información en pantalla.

Todas las variables deben tener un tipo de dato asignado que después pueda ser utilizado para realizar algún tipo de cálculo o nombramiento.

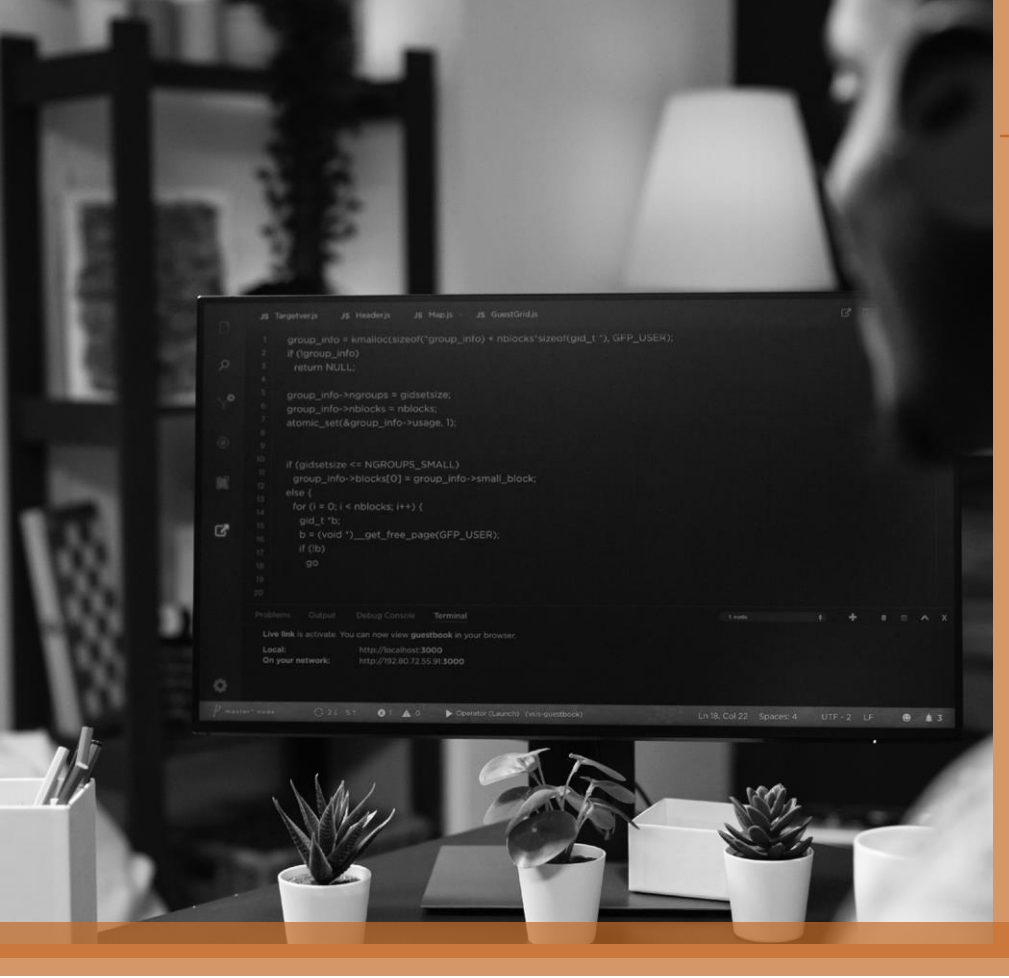


VARIABLES Y CONSTANTES

Identificador de variable. Es el nombre que define una variable. Para su creación, se debe empezar con una letra, o bien, por el carácter "_". Después, se continúa con letras o números. No obstante, el identificador debe seguir estas **normas:**

- No debe contener espacios.
- No debe comenzar por un número.
- No debe llevar ninguna palabra reservada como *if*, *while*, *for*, etcétera.





VARIABLES Y CONSTANTES

- En algunos lenguajes de programación se debe comenzar con algún carácter especial, por ejemplo “$”.

Cabe mencionar que algunos lenguajes son sensibles con el tema de mayúsculas y minúsculas (por ejemplo, el lenguaje C). Esto se conoce como ***case sensitive***.

En este contexto, para el compilador no es lo mismo el identificador `correo` que el identificador `Correo`.

Ejemplos de identificadores escritos de manera correcta e incorrecta:

Correcto	Incorrecto
color	2color
hola32	hola!
Puertp_vertical	Puerto….vertical
	char

Para definir una variable en lenguaje C, debemos poner su tipo, el identificador y, para finalizar, un punto y coma ";", por ejemplo:

```
int numero;
```

VARIABLES Y CONSTANTES

Las variables tienen dos propiedades:

- **Indirección.** Nos dice que el nombre de la variable equivale a su valor. Por ejemplo:

```
int x, y;        // declaración
x = 5;           // "x" recibe el  valor de "5"
y = x + 4        // "x" ahora también es su valor
```

VARIABLES Y CONSTANTES

- **Asignación.** Nos dice que el valor de la variable puede cambiar cuando se necesite. Por ejemplo:

```
int x, y;          // declaración
x = 5;             // “x” recibe el valor de “5”
x = x + 10;        //El valor de la variable cambia
```

VARIABLES Y CONSTANTES

Las variables pueden inicializarse desde que se declaran. Por lo anterior, es necesario que se escriban con la siguiente **estructura**:

```
"Tipo de la variable" +
"identificador" + "=" + "valor
inicial" + ";"
```

Ejemplo:

```
float numero = 12.6;
```

VARIABLES Y CONSTANTES

Existen dos lugares donde se pueden declarar variables:

1. Variables globales

2. Variables locales





- **Variables globales.** Se definen afuera del cuerpo de una función. Pueden ponerse al principio o al final del programa creado. De esta manera, cualquier función puede tener acceso a este tipo de variables donde quiera que se encuentren.

Sin embargo, no es tan viable utilizar las variables de esta forma, ya que su lectura puede ser difícil a la hora de buscarlas. Además, podría no saberse dónde se encuentran realmente, lo que causaría equivocación a la hora de invocarlas.

VARIABLES Y CONSTANTES

- **Variables locales.** Son las que fueron definidas dentro de la misma función, es decir, todas están en el bloque de código donde serán utilizadas. A diferencia de las variables globales, estas no pueden ser invocadas desde cualquier parte del programa.

Es recomendado utilizar estas variables, ya que su uso es favorable a la hora de programar. Así, las variables se encuentran en el lugar donde se utilizarán, por lo que generan menor pérdida de tiempo al localizarlas.





VARIABLES Y CONSTANTES

Existen dos formas de declarar variables:

- **Variables estáticas.** No se puede acceder a ellas desde otro bloque de código. Además, su valor se mantiene durante todo el programa, por lo que se inicializan una sola vez.

- **Variables dinámicas.** Sus valores se van calculando a lo largo de la ejecución del programa. Además, estos se mantienen hasta que la ejecución termina.

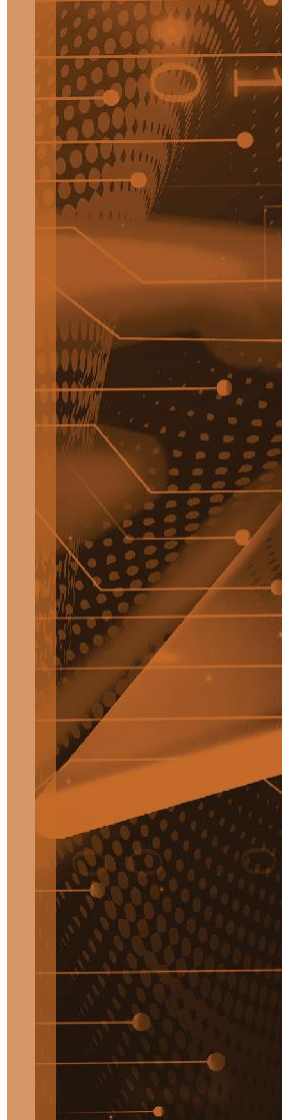
VARIABLES Y CONSTANTES

En resumen, una variable es **estática** cuando se crea **fuera** de una función; y es **dinámica** cuando se crea **dentro** de una función. Por ejemplo:

```
#include <stdio.h>
 float numero1 = 23;

main()
{
                float numero2 = 22;
                printf("%d, %d\n",
                numero1, numero2);
}

//Es estática la variable "numero1"; y
dinámica la variable "numero2".
```



VARIABLES Y CONSTANTES

Las variables se clasifican conforme a su contenido y su uso:

- **Por su contenido se dividen en:**
  - **Variables numéricas.** Son las que almacenan números, ya sean positivos, negativos o decimales. Por ejemplo:

```
salario = 1500.50;
grados = -3;
```

  - **Variables lógicas.** Son las que contienen dos valores únicamente: verdadero o falso. Por ejemplo:

```
bool var = true;
```

VARIABLES Y CONSTANTES

- **Variables alfanuméricas.** Son aquellas que están formadas únicamente por letras, números y caracteres especiales. Por ejemplo:

```
apellido = “Cruz”;
```

- **Por su uso se dividen en:**

  - **Variables de trabajo.** Son aquellas cuya función es almacenar el resultado de una operación matemática. Por ejemplo:

```
multiplicación = a / b
```

- **Contadores.** Son aquellas variables cuya función es llevar el registro del número de veces que se realiza una operación en secuencia. En otras palabras, lleva el control de cuántas veces ha pasado por ese mismo lugar. Por ejemplo:

```
contador = 0
contador = contador + 1
contador = contador + 1
```

En este caso, la variable llevó el control en dos veces.

- **Acumulador.** Son las variables que tienen como función el llevar una suma acumulativa de una serie de datos que se van dando o calculando en una operación de manera secuencial. Por ejemplo:

```
salario = 0
salario = salario + 12
salario = salario + 34
```

En este caso, la variable acumuló la cantidad del salario final, que es “46”.

VARIABLES Y CONSTANTES

- **Constante.** Es un conjunto de datos definidos que no cambian.

En el lenguaje C, el valor de una constante se va sustituyendo en todas las partes del programa donde se manda a llamar.

El uso de una constante se lleva a cabo cuando es necesario utilizar este valor almacenado en diferentes partes de un programa.

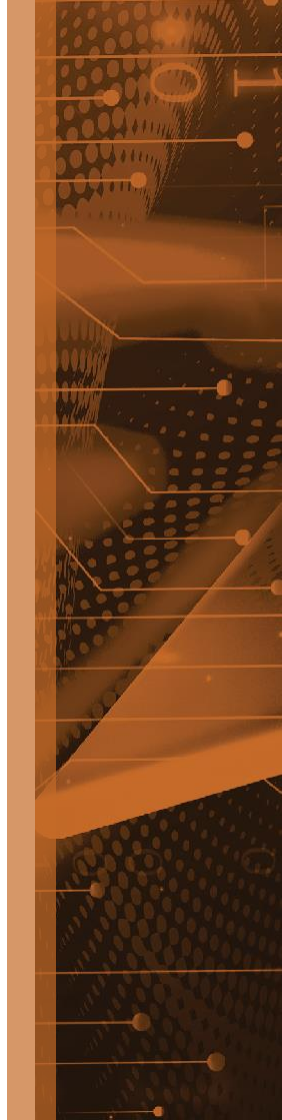


VARIABLES Y CONSTANTES

Si en algún momento requerimos reemplazar o modificar el valor de una constante, no se debe hacer en todas las partes donde fue utilizada, solo en su declaración.

Por ejemplo, si mi constante es “PI=3.1416” y queremos que ahora sea “PI=3.14” quedaría de la siguiente manera:

```
#define PI 3.1416 ó const PI = 3.1416
```



Tipos de constantes:

- **Constantes literales**. Son valores de cualquier tipo que no se declaran, ya que no tienen nombre. Por ejemplo:

```
a=b+7        //En este caso la constante
             literal es "7"
```

Algunas constantes literales más utilizadas son: constantes lógicas, de tipo entero, reales, de tipo caracter y de tipo cadena de caracteres.

- **Constantes definidas.** Se sitúan usualmente al inicio del programa, pero de igual forma se pueden definir en cualquier lugar. Por ejemplo:

```
#define numero 2012
```

El uso de constantes definidas facilita la programación, ya que normalmente las encontramos al principio del programa, lo cual permite modificarlas fácilmente.

- **Constantes de memoria.** Estas constantes utilizan un tipo de identificador de tipo de constante, así podemos indicar que los datos que contiene no se pueden modificar. Su estructura quedaría de la siguiente manera:

```
const type nombre = valor;
```

Por ejemplo:

```
const float PI = 3.1416;
```

Diferencia entre const y #define

`const` tiene un tratamiento similar al de una variable, a la que se le puede poner un tipo de dato.

`#define` indica que escribir el nombre es equivalente a su valor, el cual es un tratamiento diferente al de una variable.

El uso de `#define` ayuda cuando no se tiene el compilador que realiza la verificación de las variables. Por eso, su uso es recomendado cuando existan valores de uso frecuente en el programa.



VIDEO



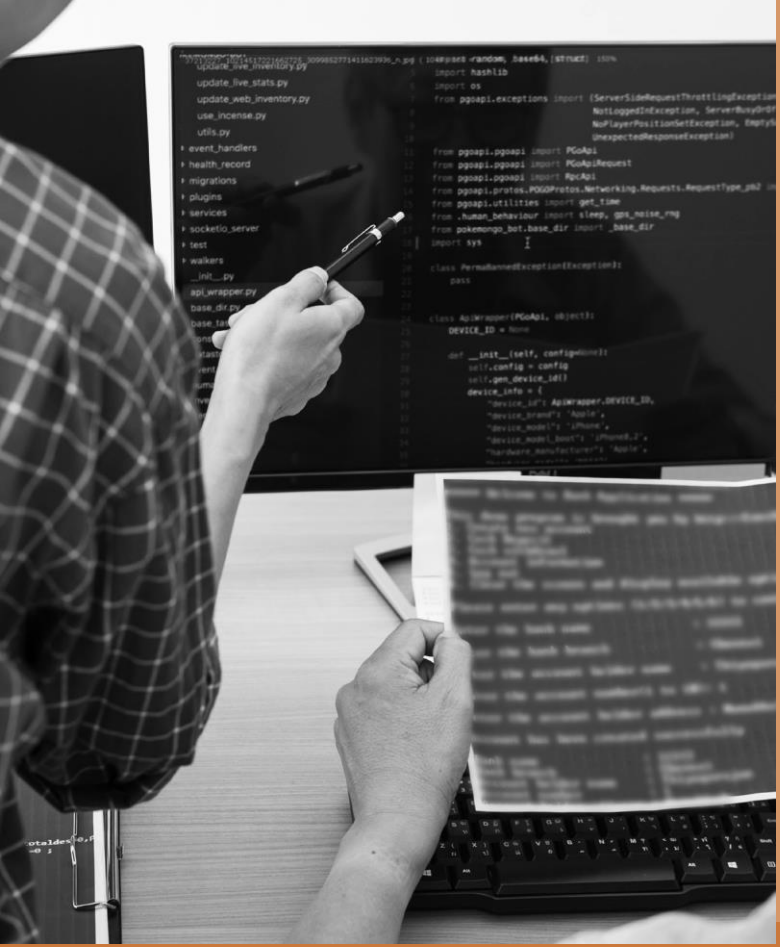
Te invitamos a ver el siguiente video:

TIPOS DE DATOS

Los **tipos de datos** son los atributos que definen el contenido de la variable. Son muy importantes a la hora de programar, ya que su correcto uso garantiza buenos resultados, así como un programa libre de errores.

El o**bjetivo** de los tipos de datos es definir qué tipo de operación realizará la variable o constante dentro del programa.





TIPOS DE DATOS

Los tipos de datos se clasifican de dos formas, a saber, en **simples** y **estructurados**:

- **Datos simples.** Se llaman así porque ocupan solo 1 byte en memoria (usualmente). Una variable de este tipo hacen referencia a un único valor a su vez. Dentro de esta clasificación se encuentran:
  - **Enteros.** Son los números que pueden tener un valor positivo, negativo o cero. No pueden contener números decimales ni fracciones. Por ejemplo: *2, -2, 0...*

TIPOS DE DATOS

En el lenguaje C, para poder declarar variables o constantes de tipo entero, se debe expresar de la siguiente manera:

- **Para constantes:**

```
const int numero = 12;
```

- **Para variables:**

```
int numero = -90;
```

De igual forma, si tenemos diferentes variables que son del mismo tipo, podemos escribirlas de esta manera:

```
int edad, numero, valor;
```

- **Reales.** Este tipo de dato hace referencia a los números reales (es decir, con decimales).

En el lenguaje C, para poder declarar variables o constantes con este tipo de dato, se debe anteponer la palabra reservada `float` al identificador de la variable, por ejemplo:

```
float valor = 12.2;
```

En el caso de las constantes, quedaría de la siguiente forma:

```
const float PI = 3.1416;
```

TIPOS DE DATOS

- **Caracteres.** Estos tipos de datos son usados para introducir cualquier número, símbolo, espacio, letra, mientras que cada carácter ocupa 1 byte de espacio.

En lenguaje C, `char` es muy similar al tipo de dato `int`, con la diferencia que su rango de almacenamiento es menor.

Para representar este tipo de dato en una variable en lenguaje C utilizamos la siguiente estructura:

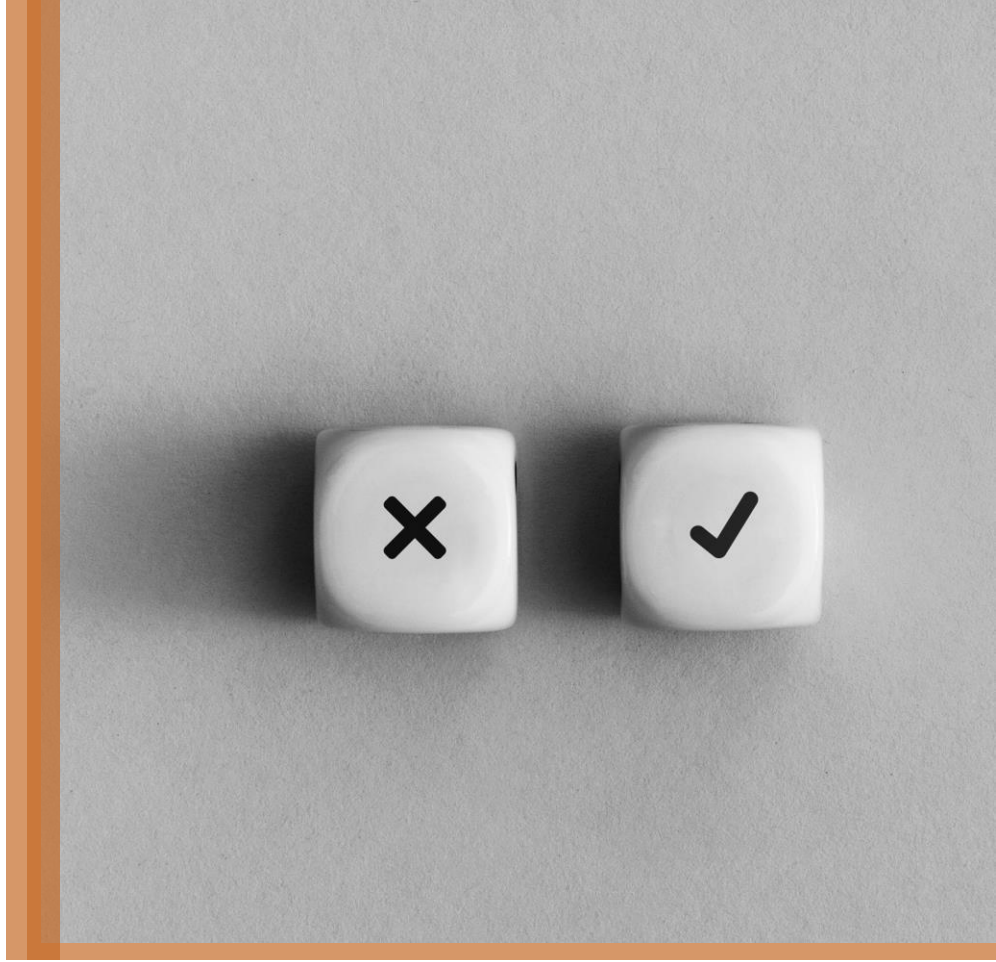
```
char nombre = “Cindy12”;
```

TIPOS DE DATOS

- **Booleanos.** Este tipo de dato solo puede ser:

    1. Verdadero y
    2. Falso.

Su función es representar una comparación entre otros datos, ya sean números o texto.





TIPOS DE DATOS

- **Datos estructurados.** Se les llama así debido a que sirven para agrupar una serie de variables de un mismo tipo con un único nombre.

Dentro de esta clasificación se encuentran, a su vez, los siguientes tipos:

- **Arreglos (*array*, matriz).** Permiten almacenar un conjunto de datos del mismo tipo. Su principal objetivo es organizar y clasificar los datos que contiene. De igual forma, facilita la búsqueda de los mismos, ahorrando al programador tiempos de creación del programa. Por ejemplo:

```
int notas [10] = {9, 2, 3, 7, 8, 7, 8, 9, 8, 8};
```

El anterior es un *array* que representa una serie de calificaciones.

- **Cadena (*string*).** Permite almacenar más de un dato, por lo que posibilita reunir varios datos a la vez. En este sentido, es muy útil para textos. Estos pueden ser: nombres de personas, ciudades, etc. De igual forma, dentro del mismo se pueden incluir números. Sin embargo, al hacerlo, se perdería su valor para ser utilizado en una operación, ya que se convertiría en texto. Por ejemplo:

```
string nombre;
```

ACTIVIDAD 1

Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad:



Presiona el botón para entregar la actividad:



CONCLUSIÓN

Durante esta unidad pudimos conocer las variables y constantes que existen. De la misma manera, vimos la forma de declararlos y emplearlos en programas. A su vez, conocimos los tipos de datos que hay (principalmente en lenguaje C), así como la función de cada uno de ellos y la forma en que se utilizan.

En definitiva, es muy importante que conozcas esta información, ya que te ayudará a crear programas de manera eficaz y rápida. Por consiguiente, serás capaz de resolver problemas computacionales de todo tipo en cualquier organización.



¡FELICIDADES!

Acabas de concluir la segunda unidad de tu curso *Introducción al Desarrollo de Software*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

UNIDAD 3

INTERACCIÓN CON EL USUARIO



TEMARIO



3.1 FUNCIONES DE ENTRADA Y SALIDA



3.2 OPERADORES Y EXPRESIONES



INTRODUCCIÓN

En esta tercera unidad aprenderás a identificar las funciones de entrada y salida. De la misma manera, comprenderás el uso de operadores y expresiones.

Esta distinción te permitirá crear programas en lenguaje C de una forma más sencilla y correcta.

COMPETENCIAS A DESARROLLAR



El alumno será capaz de definir funciones de entrada y salida.

El alumno será capaz de definir operadores y expresiones.

FUNCIONES DE ENTRADA Y SALIDA

La **librería stdio.h** (*Standard Input-output Header*) permite ingresar valores de entrada/salida.

Es la librería que va a la cabecera de un programa, ya que contiene constantes, macros y declaraciones de funciones para hacer operaciones básicas de entrada/salida.

Trabaja con caracteres que son datos o números, es decir, debe ser una cadena de caracteres.

Su **código para mandarla a llamar** es:

```
#include <stdio.h>
```





FUNCIONES DE ENTRADA Y SALIDA

Por otra parte, una **macro** es un `#define` que es capaz de realizar soluciones lógicas u operaciones matemáticas.

Las macros se utilizan para automatizar operaciones repetitivas, ayudando al programador a reutilizar código. **Ejemplo de macro:**

```
#define CUBE(A) (A)*(A)*(A)
```

En este caso, esta macro es para sacar el volumen de un cubo.

FUNCIONES DE ENTRADA Y SALIDA

Funciones de la librería stdio.h:

- *getchar,*
- *putchar,*
- *scanf,*
- *printf,*
- *gets y*
- *puts.*

Estas funciones permiten transferir información de una computadora a los dispositivos de entrada/salida estándar, como podrían ser un teclado o un monitor.



Funciones de Entrada

- **getchar.** Con el uso de esta función, puedes ingresar un carácter usando el teclado.

Sintaxis:

```
variable = getchar();
```

Ejemplo:

```
#include <stdio.h>
int main()
{
      char c;
     printf("Digita un caracter");
      c = getchar();
     …
}
```



En el anterior ejemplo podemos ver la librería `stdio.h` declarada al principio.

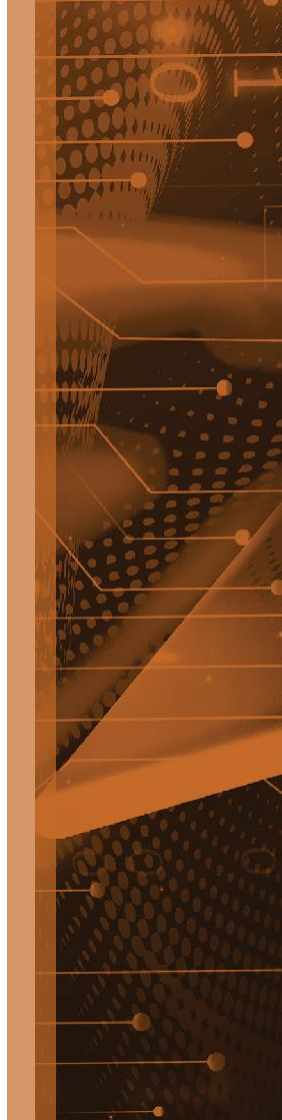
En la cuarta línea vemos que se declara la variable “c”, de tipo *char*, para que se pueda aceptar la introducción de un carácter.

En la sexta línea del programa vemos la función `getchar`, la cual permite que se pueda leer el carácter, y entonces se le asigne a la variable “c”.

- **scanf.** Esta función permite al usuario introducir datos a un programa mediante el teclado. De esta manera, se pueden añadir valores numéricos, cadena de caracteres o texto común. La función devolverá los datos ingresados pero no los mostrará por pantalla. En este sentido, con scanf podemos ingresar varios datos a la misma vez en una sola línea.

Sintaxis:

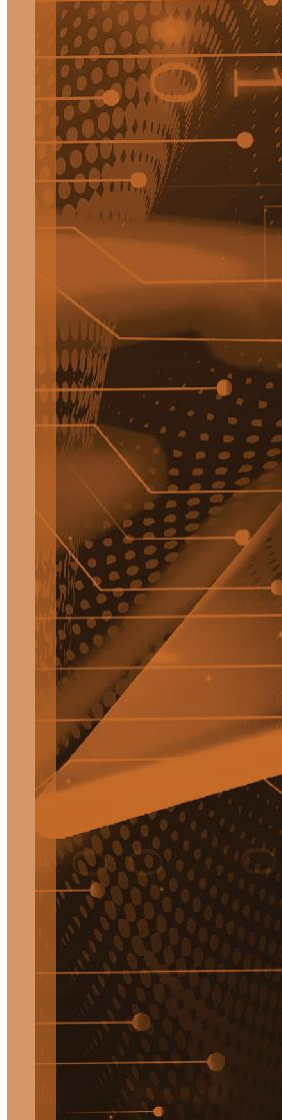
```
scanf(cadena de
control,arg1,arg2,...,argN);
```



Ejemplo:

```
#include <stdio.h>
int main()
{
                int a, b, c;
                printf( "Introduce tres
números: " );
                scanf ( "%i %i %i", &a,
&b,             &c );
                printf( "Has tecleado
los números %i %i %i\n", a,    b, c );
}
```

scanf permite introducir los tres valores del ejemplo, y **printf** los muestra en pantalla.



FUNCIONES DE ENTRADA Y SALIDA

Con scanf, cada carácter debe tener un signo "%" seguido de su carácter de conversión que señala su tipo de dato.

De igual forma, cada nombre de la variable debe ser escrito con un signo "&" antes de su nombre, excepto en los *arrays*.

Un problema con scanf es que no lee una cadena de caracteres que contiene espacios. Si ingresas un nombre completo (nombre y apellido), no se mostrará el apellido, ya que contiene un espacio entre ambos.



FUNCIONES DE ENTRADA Y SALIDA

A continuación, se muestra una tabla con los caracteres de conversión y su significado de dato más comunes y utilizados en lenguaje C:

Carácter de conversión	Significado del dato
c	Carácter
d	Entero decimal con signo
f	Flotante
s	Cadena de caracteres
i	Entero con signo

- **gets.** Esta función consiste en permitir leer una cadena de caracteres ingresada por el usuario, ya sean números o letras. Simplemente toma esa cadena y la guarda en una variable.

Sintaxis:

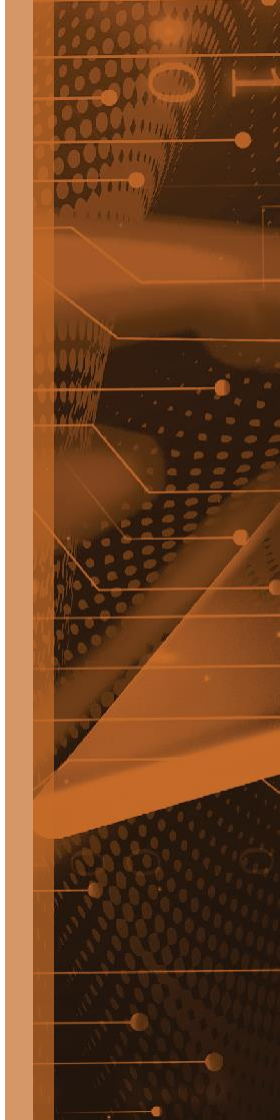
```
gets(variable);
```

Así mismo, al ingresar el valor, el usuario deberá teclear *enter* para que el flujo del programa continúe.

Ejemplo:

```
#include <stdio.h>
main()
{
                char cadena[50];
                puts("Escriba un
texto:");       gets(cadena);
                puts("El texto escrito
es:");          puts(cadena);
}
```

Observamos en el ejemplo que se crea una variable tipo *char*, en donde se guardarán los datos ingresados por el usuario. Así, gets nos permite ingresar cadenas de caracteres grandes con espacios.



Funciones de Salida

- **putchar.** Con el uso de esta función puedes conseguir un solo carácter de salida mediante el uso del monitor. Su uso suele ser muy sencillo.

Sintaxis:

```
putchar(variable);
```

Donde la palabra *variable* será alguna variable ya declarada anteriormente para utilizarla en esta parte del programa.

De esta forma, podemos ver que con la función **getchar** recibimos el contenido de la variable que debe ser un carácter. Por su parte, con **putchar** lo mostraremos en pantalla.

En este sentido, `putchar` hace su función similar a la de `printf`.

Ejemplo:

```
#include <stdio.h>
int main()
{
                char c;
                printf("Digita un caracter");
                c = getchar();
                putchar(c);
}
```

- **puts.** Esta función consiste en permitir visualizar una cadena de caracteres que haya ingresado el usuario por teclado.

Sintaxis:

```
puts(variable);
```

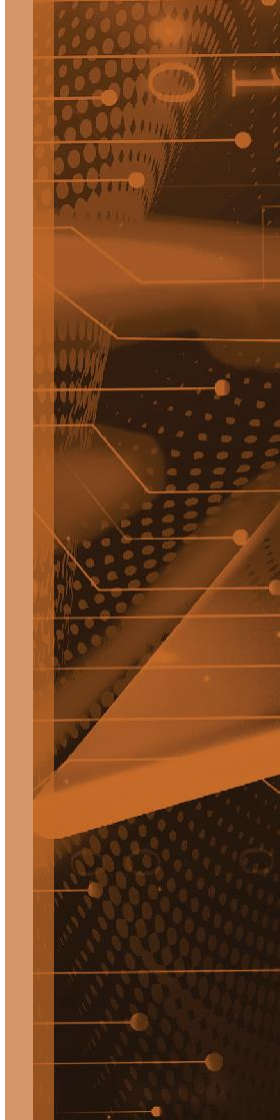
Donde *variable* deberá ser la cadena de caracteres ingresada.

De esta manera, a diferencia de scanf, puts permite mostrar la cadena de caracteres con espacios incluidos.

Ejemplo:

```
#include <stdio.h>
main()
{
                char cadena[50];
                puts("Escriba un texto:");
gets(cadena);
                puts("El texto escrito
es:");         puts(cadena);
}
```

Así, vemos que el usuario podrá visualizar los datos tal cual los ingresó en la función **gets**, pero produciendo un salto de línea.



FUNCIONES DE ENTRADA Y SALIDA

- **printf.** Con esta función se pueden imprimir datos introducidos por el usuario, ya sea texto o números.

Sintaxis:

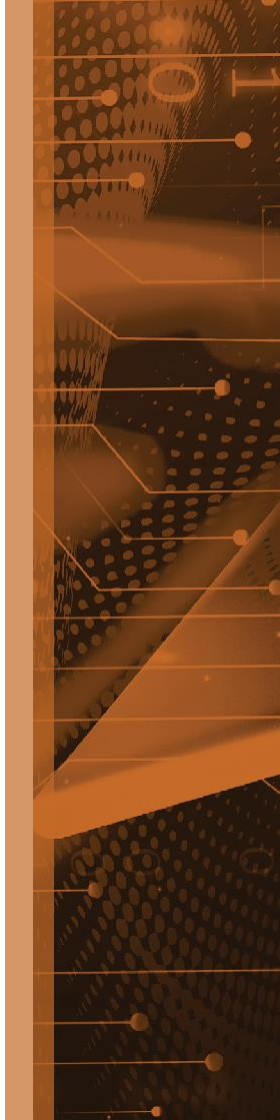
```
printf(cadena de
control,arg1,arg2,...,argN);
```

Esta es la función mas común y utilizada en lenguaje C para imprimir texto o valores introducidos por el usuario. En efecto, es la manera más sencilla de hacerlo.

Ejemplo:

```
#include <stdio.h>
int main()
{
                int n1, n2, suma;
                printf("Ingresa valor 1:
");             scanf("%d", &n1);
                printf("Ingresa valor 2:
");             scanf("%d", &n2);
                suma =     n1 + n2;
                printf( "La suma
                es: %d",    suma );
}
```

De esta manera, vemos visualizada la operación correspondiente del programa ejecutado.



VIDEO



Te invitamos a ver el siguiente video:

OPERADORES Y EXPRESIONES

Expresiones. Son instrucciones que, al momento de realizar una operación establecida, regresan una solución. En lenguaje C, la mayoría de las expresiones regresan un entero.

Existen tres tipos de expresiones:

- Valores por la izquierda
- Valores por la derecha
- Llamada a una función





OPERADORES Y EXPRESIONES

- **Valores por la izquierda.** Al evaluarlas, regresan la dirección de un dato, lo que permite acceder al mismo para realizarle modificaciones.

Son las expresiones que se colocan al lado izquierdo del "=" en una asignación.

Estos valores comúnmente son nombres de variables o elementos de un *array*.

OPERADORES Y EXPRESIONES

- **Valores por la derecha.** Al ser evaluados, se obtiene un dato de cierto tipo. Son las expresiones que se colocan al lado derecho del "=" en una asignación. Comúnmente son los contenidos de las variables.

- **Llamada a una función.** Si la función no es de tipo *void*, entonces regresará un dato. Este dato puede ser usado como una expresión del tipo *valor por la derecha*.





OPERADORES Y EXPRESIONES

Operadores. En programación, representan los símbolos que indican qué tratamiento se le dará a uno o varios operandos en una instrucción del programa.

La función de los operadores depende del tipo que se utilice en ese momento.

Los tipos de operadores más comunes son:

- **Operadores de asignación.** Son aquellos cuya función es la de asignar un valor a una variable a través de una asignación simple, o por dos operadores. A esto se le conoce *operadores compuestos*.



OPERADORES Y EXPRESIONES

En lenguaje C existen diferentes operadores de asignación, cuyo objetivo es simplificar expresiones complejas, como auto operaciones y los incrementos/decrementos.

Ejemplos:

Operador	Acción	Ejemplo	Resultado
=	Asignación Básica	X = 6	X vale 6
*=	Asigna Producto	X *= 5	X vale 30
/=	Asigna División	X /= 2	X vale 3
+=	Asigna Suma	X += 4	X vale 10
-=	Asigna Resta	X -= 1	X vale 5

Operador	Acción	Ejemplo	Resultado
%=	Asigna Módulo	X %= 5	X vale 1
<<=	Asigna Desplazamiento Izquierda	X <<= 1	X vale 12
>>=	Asigna Desplazamiento Derecha	X >>= 1	X vale 3
&=	Asigna AND entre Bits	X &= 1	X vale 0
^=	Asigna XOR entre Bits	X ^= 1	X vale 7
|=	Asigna OR entre Bits	X |= 1	X vale 79

Nota. Los últimos cinco operadores necesitan tener un tipo de dato específico (por ello se utiliza *int*). El resto puede utilizar cualquiera, mientras no sea una constante.

OPERADORES Y EXPRESIONES

- **Operadores aritméticos.** Son aquellos cuya función es permitir hacer operaciones o cálculos simples. En este tipo de operadores encontramos los llamados operadores de decremento e incremento, los cuales, según su posición, cambian el resultado de la operación. Por ejemplo, si el operador se encuentra antes de la variable se conoce como **prefijo**, por lo cual realiza primero el incremento o decremento y enseguida utiliza el valor asignado en la variable.



Por otra parte, si el operador está después, se conoce como **posfijo**. Entonces, primero utiliza el valor asignado en la variable y después realiza el decremento o incremento.

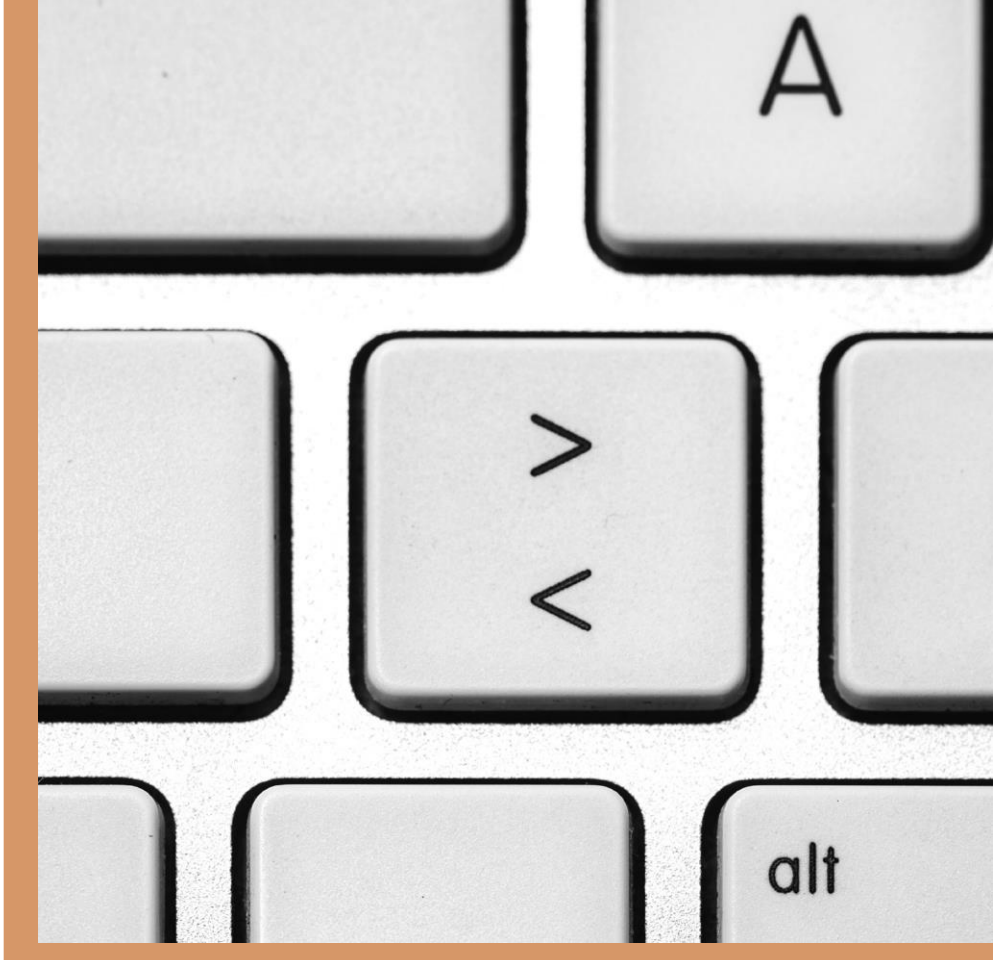
Ejemplos:

Operador	*Acción*	*Ejemplo*
–	Resta	x = 5 – 3; // x vale 2
+	Suma	x = 2 + 3; // x vale 5
*	Multiplicación	x = 2 * 3; // x vale 6
/	División	x = 6 / 2; // x vale 3
%	Módulo	x = 5 % 2; // x vale 1
--	Decremento	x = 1; x -- // x vale 0
++	Incremento	x = 1; x++ // x vale 2

OPERADORES Y EXPRESIONES

- **Operadores relacionales o de comparación.** Son aquellos cuya función es permitir evaluar una expresión. Por ejemplo, se encargan de conocer si dos variables son iguales, diferentes, o si una es menor o mayor que la otra. A su vez, permite identificar si son de tipo *booleano* o no.

Este tipo de operadores se utilizan más para estructuras de control donde se deben evaluar expresiones.



Ejemplos:

Operador	*Acción*	*Ejemplo*
<	Menor	x = 5 y=3; (y < x)
>	Mayor	x =7 y=9; (y > x)
<=	Menor o igual	x = 5 y= 3; (y <= x)
>=	Mayor o igual	x =7 y=5; (x >= y)
==	Igual	x = 5 y =5; (x == y)
i=	Diferente	x=9 y=5; (x i= y)

Se debe tener **mucho cuidado con no confundir** el operador de asignación "=" con el operador relacional "==".

- **Operadores lógicos.** Son aquellos cuya función es producir un resultado *booleano*, ya sea falso o verdadero; "1" o "0".

Ejemplos:

Operador	*Acción*	*Ejemplo*
&&	Conjunción (Y) and	C && D
||	Disyunción (O) or	C|| D
!	Negación	C!

Cabe destacar que, para estudiar su resultado, se utilizan las tablas de verdad.

ACTIVIDAD 2

Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad:



Presiona el botón para entregar la actividad:



CONCLUSIÓN

En esta unidad pudimos conocer la manera en que se puede interactuar con el usuario. De la misma manera, comprendimos las funciones de entrada/salida, que permiten al usuario ingresar datos, y a la computadora, darle una respuesta.

Además, comprendimos la utilización de operadores y expresiones que ayudan a que la comunicación sea eficaz. Por consiguiente, permiten desarrollar problemas más complejos con ayuda de programas.



¡FELICIDADES!

Acabas de concluir la tercera unidad de tu curso *Introducción al Desarrollo de Software*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

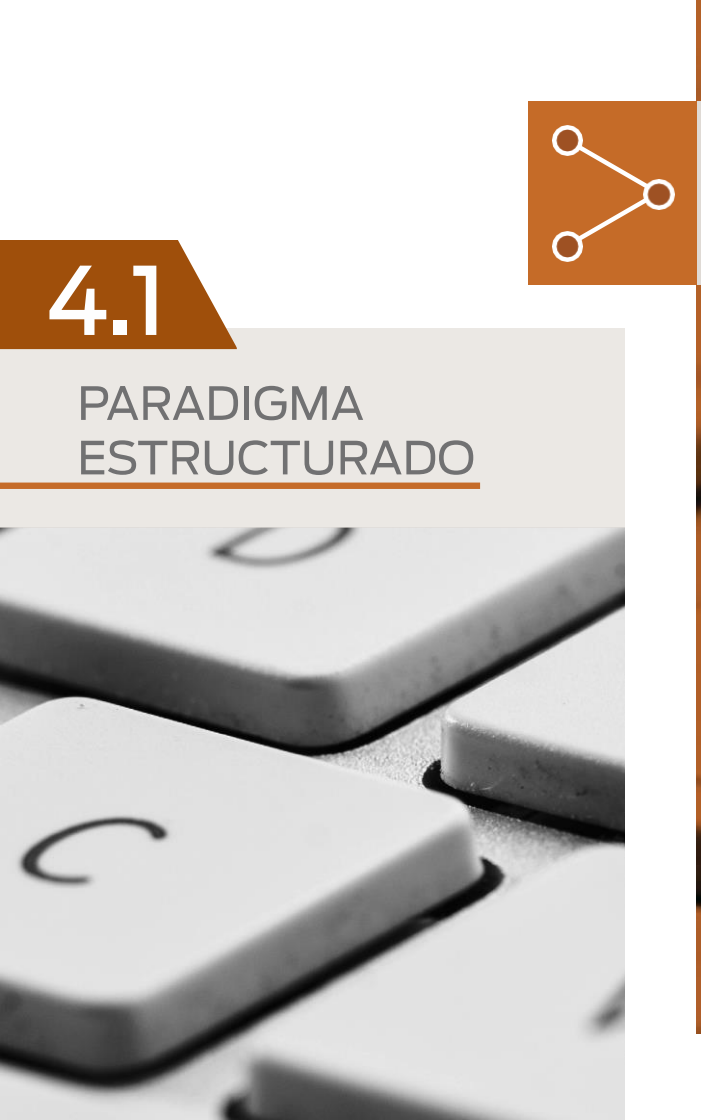
4

UNIDAD

PARADIGMAS DE PROGRAMACIÓN



TEMARIO



4.1 PARADIGMA ESTRUCTURADO



4.2 PARADIGMA ORIENTADO A OBJETOS



INTRODUCCIÓN

En esta cuarta unidad aprenderás dos distintos paradigmas de programación, es decir, el estructurado y el orientado a objetos.

Estos conocimientos te facilitarán el diseño de proyectos más cómodos para el usuario final, ya que permiten la creación de interfaces más sofisticadas y eficientes.

COMPETENCIAS A DESARROLLAR



El alumno será capaz de identificar los paradigmas estructurados.

El alumno será capaz de identificar los paradigmas orientados a objetos.

PARADIGMA ESTRUCTURADO

Paradigma de programación. Es un conjunto de métodos de programación de software que se encarga de clasificar a todos los lenguajes de programación según sus características.

Cada lenguaje de programación tiene uno o varios paradigmas según los tipos de órdenes que aceptan. En este caso hablaremos de **dos**:

- Paradigma Estructurado y
- Paradigma Orientado a Objetos.





PARADIGMA ESTRUCTURADO

Paradigma Estructurado. Es un paradigma de programación que se centra en mejorar la calidad y tiempo de desarrollo de un programa. Se caracteriza por no utilizar la instrucción *goto*.

Este paradigma pretende resolver un problema de inicio a fin en una sola estructura o bloque de código. En este sentido, sigue el método de **programación imperativa**, el cual se basa en dar una serie de instrucciones a la computadora sobre cómo hacer las tareas en forma de algoritmos.

PARADIGMA ESTRUCTURADO

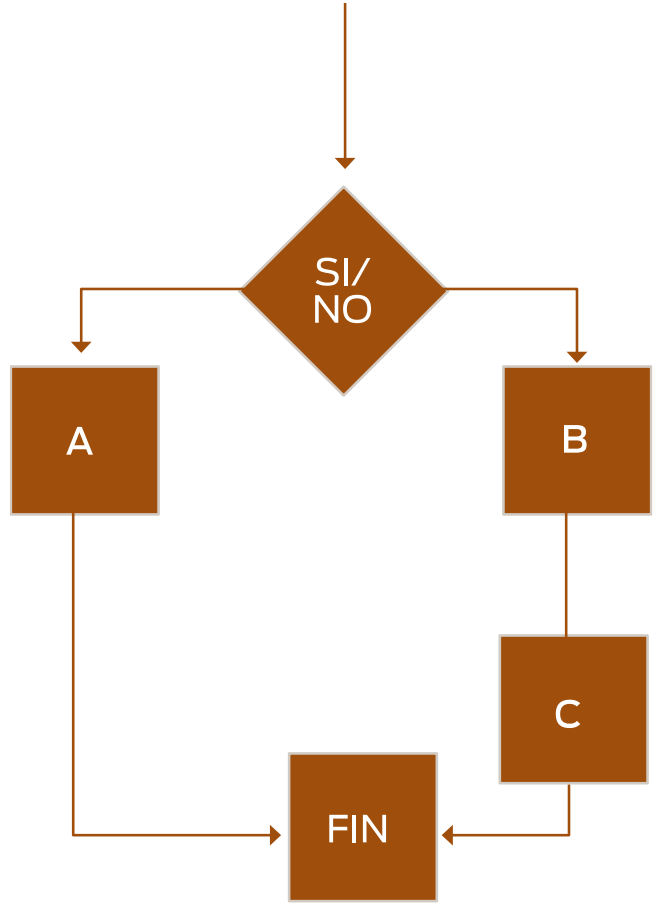
Este paradigma utiliza **tres estructuras de control**:

- **Secuencial.** Conformada por llamadas a instrucciones o funciones. Por ejemplo:



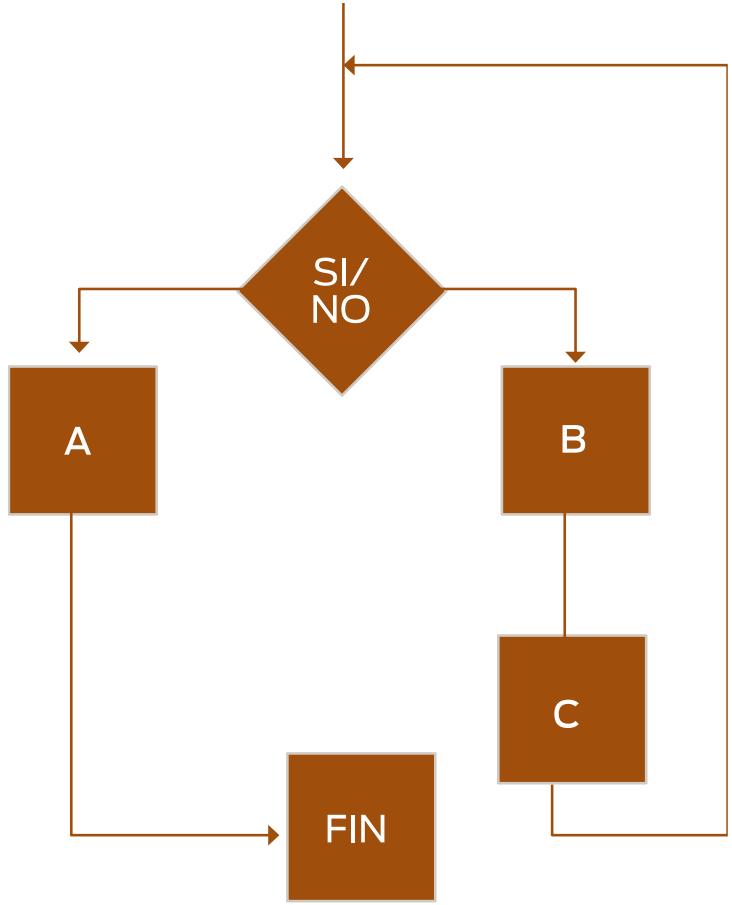
PARADIGMA ESTRUCTURADO

- **Condicional.** Ejecuta un trozo de código si se cumple una condición *booleana*. Por ejemplo:



PARADIGMA ESTRUCTURADO

- **Repetitiva.** Ejecuta un trozo de código hasta que se cumpla una condición *booleana*. Por ejemplo:



PARADIGMA ESTRUCTURADO

Ventajas del paradigma estructurado:

- Facilita la comprensión de la tarea que realiza una función, ya que todas las instrucciones están encerradas en un bloque de código.

- No hay necesidad de buscar cada salto de línea (*goto*) dentro de los trozos de código para intentar entender la lógica.

- Ayuda a optimizar los tiempos de las fases de prueba y depuración.





PARADIGMA ESTRUCTURADO

Características del paradigma estructurado:

- Puede ser leído en secuencia, desde el inicio hasta el final del programa, sin que se pierda la continuidad de la instrucción que se debe cumplir.

- Se incrementa el rendimiento de los programadores, en comparación con la utilización de la instrucción "*goto*".

- Los programas son mucho más fáciles de crear.

VIDEO



Te invitamos a ver el siguiente video:

PARADIGMA ORIENTADO A OBJETOS

Paradigma Orientado a Objetos. Es el más utilizado en la actualidad. Permite que los objetos operen a los datos que se ingresan, con el fin de lograr obtener datos de salida específicos.

Además, dichos objetos muchas veces se encuentran ya prediseñados e incluidos en las bibliotecas del lenguaje de programación que se vaya a utilizar.





PARADIGMA ORIENTADO A OBJETOS

Este paradigma se basa en los siguientes conceptos:

- **Clase.** Es un *molde* para la creación de objetos de datos según un modelo ya establecido.

  Los datos de una clase pueden ser guardados en una **estructura**.

  Cada clase es un **conjunto de variables y métodos** que se utilizan para operar con objetos, definiendo cómo y de qué tipo serán.

PARADIGMA ORIENTADO A OBJETOS

A cada objeto que es creado desde una clase se le llama **instancia de la clase**.

Ejemplo de clase:

```
Coche                    // Nombre de la clase
Arrancar, ir, parar      // Métodos
velocidad, color        // Atributos o datos
```





PARADIGMA ORIENTADO A OBJETOS

- **Objetos.** Están constituidos por datos almacenados e instrucciones que son realizadas en el momento de la ejecución del programa.

Los objetos en la POO tienen correspondencia con los objetos reales del mundo en sí; o bien, con objetos que contiene el sistema del programa.

PARADIGMA ORIENTADO A OBJETOS

Ejemplo de objeto:

```
Objeto: Ferrari
coche.ferrari              //nombre del objeto
Arrancar, ir, parar  // métodos
Velocidad, color           //atributos o datos
```



PARADIGMA ORIENTADO A OBJETOS

- **Métodos.** Son algoritmos asociados a un objeto. En otras palabras, representan lo que el objeto puede hacer.

- **Atributos.** Son las características que tiene cada clase en sí.

- **Evento.** Suceso que ocurre cuando el sistema maneja un objeto.





PARADIGMA ORIENTADO A OBJETOS

El POO se basa en los siguientes **4 principios:**

1. **Herencia.** Técnica que permite derivar una clase a otra clase. Por ejemplo:

   El nodo padre hereda al nodo hijo, y así sucesivamente.

   En la programación, la herencia es muy útil, ya que permite definir nuevas clases basándose en las ya existentes. Esto se traduce en la posibilidad de reutilizar código.

   En cambio, en el lenguaje C no ayuda mucho, por lo que es mejor utilizar una clase en lugar de heredar de ella.

2. **Encapsulación.** Contiene la información más importante de un objeto. Además, solo mostrará la información que sea seleccionada.

En la encapsulación, cada objeto es responsable de sus datos y estado. Por ello, los atributos internos de un objeto deben ser inaccesibles desde afuera. Entonces, no se podrán modificar a menos que sean llamados por una función.





3. **Abstracción.** Consiste en seleccionar información y mostrar solo lo más importante de ella. Por lo tanto, dejará a un lado los elementos del resto de información.

   Gracias a la abstracción, se facilita el mantenimiento de un código de gran tamaño.

   En otras palabras, no es necesario conocer a profundidad todos los detalles del objeto para saber cómo funciona.

4. **Polimorfismo.** Consiste en diseñar un objeto para después poder compartir sus comportamientos. Es decir, que un objeto pueda tener distintas clases. Por lo que quizás varios objetos tendrán la misma interfaz.

   El polimorfismo permite la ejecución de distintos comportamientos de dos formas:

   - Anulación del método (*overriding*) y
   - Sobrecarga del método.





- **Anulación del método (*overriding*).** Es una característica que permite que una subclase proporcione una implementación en particular de un método que ya ha sido equipado por una de sus subclases.

- **Sobrecarga del método.** Se refiere a la creación de distintos métodos con un mismo nombre, pero con distintos tipos de parámetros.

PARADIGMA ORIENTADO A OBJETOS

Ventajas del paradigma orientado a objetos:

- Permite reutilizar código, lo que genera menor pérdida de tiempo.
- Protege la información gracias al encapsulamiento.
- Se pueden construir sistemas más complejos; o sencillos y específicos, gracias a la abstracción.
- Permite corregir errores de código de manera más rápida y eficaz.



Ejemplo de paradigma orientado a objetos:

CLASE	OBJETOS	ATRIBUTO/DATOS
Empleado	Juan Pérez	Edad: 25 Puesto: Obrero
Empleado	María Castillo	Edad: 56 Puesto: Gerente



PARADIGMA ORIENTADO A OBJETOS

La principal diferencia entre el Paradigma Estructurado y el Paradigma Orientado a Objetos es que en el primero se dividen las funciones de los datos, mientras que en el segundo se definen conjuntos de objetos, y los datos son combinados de forma modular con las funciones.

CONCLUSIÓN

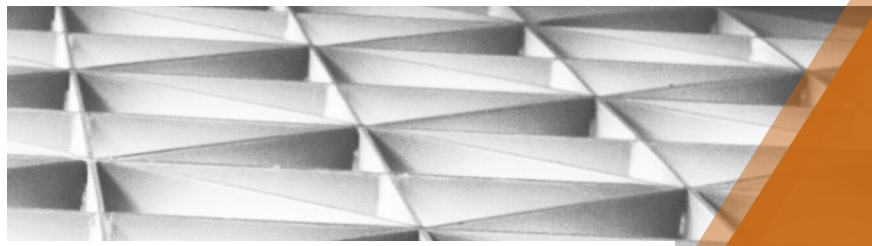
Durante esta unidad pudimos conocer más acerca del Paradigma Estructurado y el Orientado a Objetos. Es necesario comprender que cada uno tiene sus ventajas y desventajas, así como su grado de utilidad según los requerimientos de programación.

No obstante, el Paradigma Orientado a Objetos es el que actualmente se utiliza más. Esto se debe, principalmente, a que permite brindar una mejor interfaz al usuario; y al programador le ofrece gran comodidad a la hora de ir escribiendo el código.



¡FELICIDADES!

Acabas de concluir la cuarta unidad de tu curso *Introducción al Desarrollo de Software*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.



PROYECTO FINAL

PROYECTO FINAL

Te invitamos a realizar el siguiente proyecto final:

Presiona el botón para descargar el proyecto final:



Presiona el botón para entregar el proyecto final:

